CS281

Virtual Development Board

Technical Reference Manual

RISC-V RV32IM Educational Platform

Version 1.0 — Drexel University — CS281 Computer Architecture

Table of Contents

1. Introduction

The CS281 Virtual Development Board is a RISC-V RV32IM microcontroller platform emulated in Renode. It provides a realistic, bare-metal programming environment without requiring physical hardware. The board is purpose-built for the CS281 Computer Architecture course at Drexel University.

Students write programs in RISC-V assembly (and optionally C) that run directly on the emulated processor. Every peripheral is accessed through memory-mapped registers at fixed addresses, exactly as on a physical chip. The Renode simulator executes unmodified ELF binaries and exposes GDB-based debugging so students can pause execution, inspect registers, and step through code.

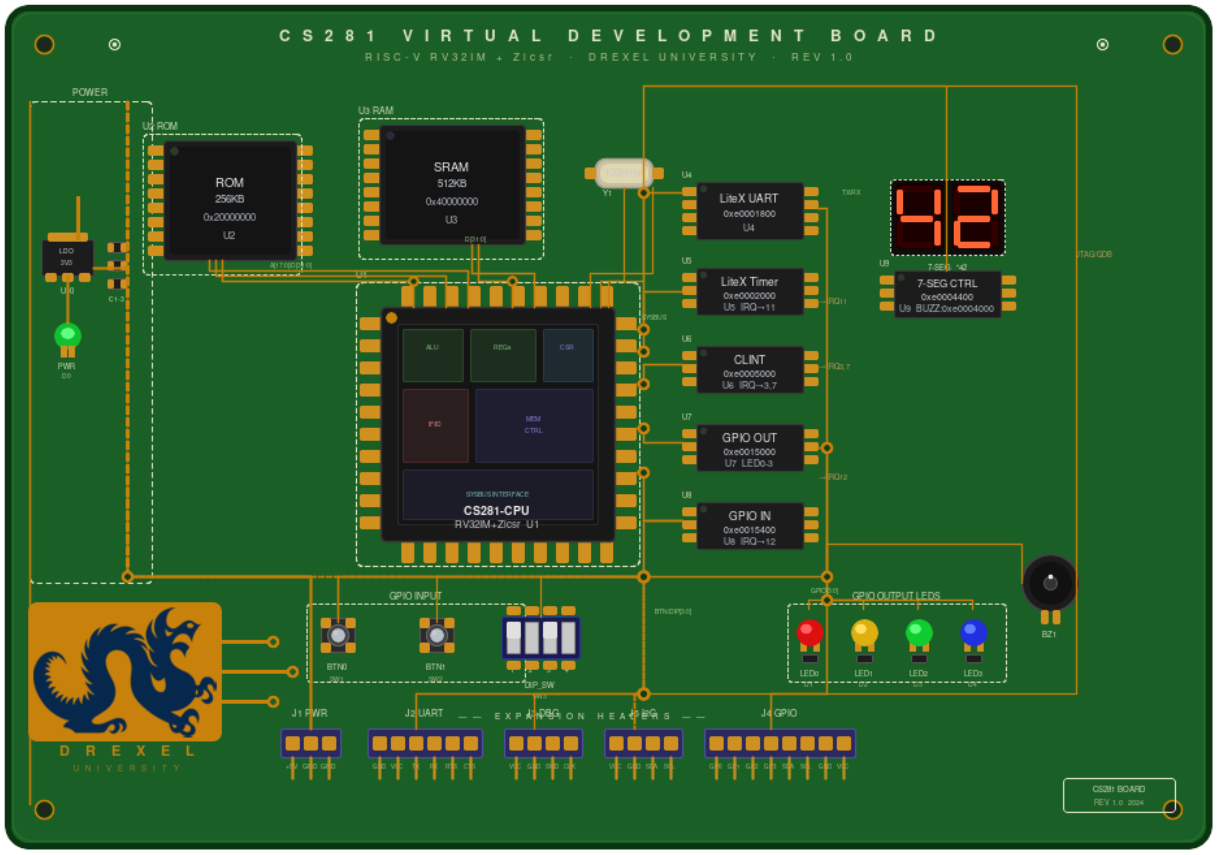

Figure 1 — CS281 Virtual Development Board (PCB rendering)

1.1 Processor Overview

Parameter	Value
Architecture	RISC-V (RV32IM + Zicsr)
Data width	32-bit
Extensions	M — integer multiply/divide; Zicsr — CSR instructions
Privilege level	Machine mode (M-mode) only
Byte order	Little-endian

Parameter	Value
Reset address	Set by ELF entry point (_start at 0x20000000)
Interrupt model	CLINT (software + timer) + external IRQs via cpu @ n

NOTE *RV32I instructions always take priority in decoding. The M extension adds mul, mulh, div, divu, rem, remu. The Zicsr extension adds csrr, csrw, csrrs, etc. Compressed (C) and Atomic (A) extensions are NOT present; do not use them.*

2. Memory Map

All addresses are physical addresses on the system bus. The processor accesses every peripheral through normal load/store instructions (lw, sw, lb, sb, etc.) to the addresses listed below.

Base Address	Name	Size	Access	Description
`0x20000000`	ROM (Flash)	256 KB	R / X	Program code and read-only data. Loaded from ELF at startup. Treated as read-only by the linker.
`0x40000000`	RAM	512 KB	R / W / X	Initialized data (.data), zero-initialized data (.bss), heap, and stack.
`0xe0001800`	UART	32 bytes	R / W	LiteX serial UART — main console I/O.
`0xe0002000`	Timer	32 bytes	R / W	LiteX countdown timer with interrupt output.
`0xe0004000`	Buzzer GPIO	4 bytes	W	Single-bit GPIO output driving the board buzzer.
`0xe0004400`	7-Segment GPIO	4 bytes	W	8-bit GPIO output driving the 7-segment display (segments a–g + dp).
`0xe0005000`	CLINT	64 KB	R / W	Core-Level Interruptor: machine software (msip) and machine timer (mtime / mtimecmp).
`0xe0015000`	GPIO Output	4 bytes	W	Four LED outputs (bits 0–3 = LED0–LED3).
`0xe0015400`	GPIO Input	4 bytes	R	Four board inputs: BTN0, BTN1, DIP_SW0, DIP_SW1 (bits 0–3).

2.1 Memory Sections

The linker script (hardware/lib/cs281.ld) maps program sections to the memory regions above. Understanding this layout is essential for writing correct linker scripts and startup code.

ELF Section	Region	Description
.text.startup	ROM (first)	startup.S — must be at the very start of ROM so _start lands at 0x20000000.
.text	ROM	All other compiled/assembled code.

ELF Section	Region	Description
.rodata	ROM	String literals and other read-only constants.
.data	ROM → RAM	Initialized global/static variables. Stored (LMA) in ROM after .rodata; copied to RAM (VMA) by startup.S at boot.
.bss	RAM	Zero-initialized globals and statics. Not stored in ROM; startup.S writes zeros to this region.
(stack)	RAM (top)	Stack grows downward from _stack_top = 0x40080000 (top of RAM). No explicit section — the linker exports the symbol only.

3. Boot Sequence and Programmer's Model

When a program is loaded into Renode (via sysbus LoadELF), the simulator sets the CPU program counter to the ELF entry point. The linker script ensures this is _start, the first instruction of startup.S, located at 0x20000000.

3.1 Startup Sequence (startup.S)

The startup file in hardware/lib/startup.S performs four steps before calling main():

Step	Action	Details
1	Set stack pointer	la sp, _stack_top loads 0x40080000 into sp. The stack grows downward into RAM.
2	Copy .data to RAM	Reads from ROM (LMA = _etext) and writes to RAM (VMA = _sdata … _edata). Required because .data has initial values that must be in RAM at runtime.
3	Zero .bss	Writes zero words from _sbss to _ebss. The C standard requires zero-initialised globals; the ELF does not store zeros, so startup must do it.
4	Call main()	call main. If main() returns (it should not), startup spins in an infinite loop (j .Lhalt).

3.2 Linker Symbols

These symbols are defined by cs281.ld and used by startup.S. They are available in any assembly or C file.

Symbol	Value	Meaning
_etext	(end of .rodata in ROM)	LMA (load address) of the .data section — where .data is stored in ROM.
_sdata	0x40000000 + offset	VMA (run address) start of .data in RAM.
_edata	0x40000000 + offset	VMA end of .data in RAM.
_sbss	(after .data in RAM)	Start of .bss region.
_ebss	(after .bss in RAM)	End of .bss region.

Symbol	Value	Meaning
_stack_top	0x40080000	Top of RAM — initial value of the stack pointer.

NOTE *_etext marks the boundary between code+rodata and the stored .data image in ROM. It is NOT simply the end of .text — it is the end of all ROM content.*

4. Interrupt Reference

The CS281 board uses RISC-V machine-mode (M-mode) interrupts. The CPU's mstatus.MIE bit globally enables interrupts; mie controls individual sources. When an interrupt fires, the CPU jumps to the address in mtvec and the mcause register identifies the source.

CPU Line	mcause	Source	Type	Clear Procedure
3	0x80000003	CLINT MSIP	Machine Software	Write 0 to CLINT_BASE + 0x0000 (MSIP register).
7	0x80000007	CLINT MTIP	Machine Timer	Write a new value to MTIMECMP greater than the current MTIME.
11	0x8000000B	LiteX Timer	Machine External	Write 1 to Timer EV_PENDING (offset 0x18) to acknowledge.
12	0x8000000C	GPIO Input	Machine External	Read GPIO_IN to acknowledge. Change must be observed; no separate pending register.

4.1 Enabling Interrupts (Assembly)

Minimal sequence to enable machine-mode interrupts in assembly:

```
    # Write the trap vector address to mtvec
    la   t0, trap_handler
    csrw mtvec, t0

    # Enable individual interrupt sources in mie
    # Bit  3 = MSIE (software), bit 7 = MTIE (timer), bit 11 = MEIE (external)
    li   t0, (1 << 11) | (1 << 7) | (1 << 3)
    csrw mie, t0

    # Set mstatus.MIE (bit 3) to globally enable interrupts
    li   t0, (1 << 3)
    csrs mstatus, t0
```

4.2 Trap Handler Template

```
trap_handler:
```

```
# Save caller-saved registers to the stack
addi sp, sp, -64
sw   ra,  0(sp)
sw   t0,  4(sp)
# ... save t1-t6, a0-a7 as needed ...

# Read mcause to identify the interrupt
csrr t0, mcause
# bit 31 = 1 for interrupts, 0 for exceptions
# bits 30:0 = interrupt/exception code (see table above)

# ... dispatch to handler ...

# Restore and return
lw   ra,  0(sp)
lw   t0,  4(sp)
addi sp, sp, 64
mret            # return to interrupted instruction
```

5. UART — Serial Console

The UART peripheral provides a bidirectional serial character interface. It is the primary means of text I/O. In Renode it is connected to a telnet socket on port 3456 (run make uart-connect in a second terminal to view output).

Base Address: 0xe0001800

Offset	Register	Access	Reset	Description
0x00	RXTX	R / W	—	Write: transmit one byte (character). Read: receive one byte.
0x04	TXFULL	R	0	1 = TX FIFO is full; do not write to RXTX until 0.
0x08	RXEMPTY	R	1	1 = RX FIFO is empty; no data ready to read.
0x0C	EV_STATUS	R	0	Raw interrupt status flags (TX ready, RX available).
0x10	EV_PENDING	R / W	0	Interrupt pending flags. Write 1 to a bit to clear it.
0x14	EV_ENABLE	R / W	0	Interrupt enable mask. Set bit to route event to the CPU.

5.1 Transmit a Character (Polling)

```
    li   t0, 0xe0001800      # UART base
.wait_tx:
    lw   t1, 4(t0)           # read TXFULL
    bnez t1, .wait_tx        # spin until space available
    sw   a0, 0(t0)           # write character (in a0) to RXTX
```

5.2 Receive a Character (Polling)

```
    li   t0, 0xe0001800      # UART base
.wait_rx:
    lw   t1, 8(t0)           # read RXEMPTY
    bnez t1, .wait_rx        # spin until byte arrives
    lw   a0, 0(t0)           # read character into a0
```

NOTE *The library function uart_putchar(a0) and uart_puts(a0) in hardware/lib/uart.S implement polling transmit. Link uart.S into your project and call them directly.*

6. Timer

The hardware timer is a 32-bit countdown timer running at 100 MHz. It fires an interrupt (CPU line 11) when it counts down to zero. Use it for precise timing instead of busy-wait software delays.

Base Address: 0xe0002000 | Frequency: 100,000,000 Hz | IRQ: CPU line 11

Offset	Register	Access	Description
0x00	LOAD	R / W	Load value. The timer counts down from this value when enabled.
0x04	RELOAD	R / W	Auto-reload value. On reaching zero the timer reloads from this register if non-zero; write 0 for one-shot mode.
0x08	EN	R / W	Timer enable. Write 1 to start, 0 to stop.
0x0C	UPDATE_VALUE	W	Write 1 to latch the current count into VALUE. Must be done before reading VALUE.
0x10	VALUE	R	Latched current count. Read after writing 1 to UPDATE_VALUE.
0x14	EV_STATUS	R	Raw timer interrupt status.
0x18	EV_PENDING	R / W	Timer interrupt pending. Write 1 to clear (acknowledge) in your ISR.
0x1C	EV_ENABLE	R / W	Write 1 to route timer interrupt to the CPU (enables CPU line 11).

6.1 One-Shot Timer Example (500 ms at 100 MHz)

```
    li   t0, 0xe0002000
    li   t1, 50000000          # 100 MHz / 2 = 0.5 s
    sw   t1, 0(t0)             # LOAD
    sw   zero, 4(t0)           # RELOAD = 0 (one-shot)
    li   t1, 1
    sw   t1, 0x1C(t0)          # EV_ENABLE
    sw   t1, 0x08(t0)          # EN — start timer
```

NOTE *To use the timer with interrupts, also set mie bit 11 (MEIE) and set mstatus.MIE as shown in Section 4.1. In your ISR, write 1 to EV_PENDING (offset 0x18) to clear the pending flag before returning.*

7. CLINT — Core-Level Interruptor

The CLINT provides machine-level timer and software interrupts directly to the CPU, per the RISC-V privilege specification. It does NOT pass through an external interrupt controller.

Base Address: 0xe0005000 | Frequency: 100,000,000 Hz

Offset	Register	Width	Access	Description
0x0000	MSIP	32-bit	R / W	Machine software interrupt pending. Write 1 to assert, 0 to clear. Fires IRQ on CPU line 3.
0x4000	MTIMECMP	64-bit	R / W	Machine timer compare. When MTIME >= MTIMECMP, asserts IRQ on CPU line 7. Write a

Offset	Register	Width	Access	Description
				new future value to clear the interrupt.
0xBFF8	MTIME	64-bit	R	Current machine time in timer ticks (100 MHz). Always increasing; not writable.

7.1 Set a Timer Interrupt 1 Second from Now

```
    li   t0, 0xe0005000
    # Read current MTIME (low 32 bits)
    lw   t1, 0xBFF8(t0)
    # Add 100,000,000 ticks (1 second at 100 MHz)
    li   t2, 100000000
    add  t1, t1, t2
    # Write to MTIMECMP (low word; high word = 0 for simplicity)
    sw   t1,  0x4000(t0)
    sw   zero, 0x4004(t0)
```

NOTE *Always write MTIMECMP high word before low word when using 64-bit compare to avoid spurious interrupts during the two-word write.*

8. GPIO Output — LEDs

The GPIO Output peripheral drives four on-board LEDs. A single 32-bit register controls all four outputs simultaneously using a bitmask. Bits 31:4 are ignored.

Base Address: 0xe0015000

8.1 DATA Register (offset 0x00)

Bits	Name	Description
31:4	—	Reserved. Always write 0.
3	LED3	1 = LED3 ON, 0 = LED3 OFF
2	LED2	1 = LED2 ON, 0 = LED2 OFF
1	LED1	1 = LED1 ON, 0 = LED1 OFF
0	LED0	1 = LED0 ON, 0 = LED0 OFF

8.2 Usage Examples

```
    li   t0, 0xe0015000
    li   t1, 0x1          # turn ON LED0 only
    sw   t1, 0(t0)
```

```
    li   t1, 0x5          # turn ON LED0 + LED2  (bits 0 and 2)
    sw   t1, 0(t0)

    sw   zero, 0(t0)      # turn OFF all LEDs
```

NOTE *Writing a new value replaces the entire register. To set a single LED without disturbing others, read-modify-write: lw t2,0(t0) / or t2,t2,t1 / sw t2,0(t0).*

9. GPIO Input — Buttons and DIP Switches

The GPIO Input peripheral reads four digital inputs: two push buttons and two DIP switch positions. Any state change on any input triggers an interrupt on CPU line 12.

Base Address: 0xe0015400 | IRQ: CPU line 12 (state change)

9.1 DATA Register (offset 0x00) — Read Only

Bits	Name	Active	Description
31:4	—	—	Reserved. Reads as 0.
3	DIP_SW1	1	DIP switch 1 position. 1 = switch ON, 0 = switch OFF.
2	DIP_SW0	1	DIP switch 0 position. 1 = switch ON, 0 = switch OFF.
1	BTN1	1	Push button 1. 1 = pressed, 0 = released.
0	BTN0	1	Push button 0. 1 = pressed, 0 = released.

9.2 Polling Example — Wait for BTN0

```
    li   t0, 0xe0015400
.wait_btn:
    lw   t1, 0(t0)         # read GPIO_IN
    andi t1, t1, 0x1       # mask BTN0 (bit 0)
    beqz t1, .wait_btn     # loop until pressed
```

9.3 Driving Inputs from the Renode Monitor

GPIO inputs have no physical hardware; they are driven from the Renode monitor console. Connect to the monitor terminal (the terminal where Renode runs) and type:

```
    gpio_in SetGPIO 0 true     # press BTN0
    gpio_in SetGPIO 0 false    # release BTN0
    gpio_in SetGPIO 2 true     # turn DIP_SW0 ON
```

NOTE *Pin numbers: 0 = BTN0, 1 = BTN1, 2 = DIP_SW0, 3 = DIP_SW1.*

10. 7-Segment Display

The 7-segment display is driven by an 8-bit GPIO output register. Each bit independently controls one segment of the display. Digits are formed by writing the correct bitmask for the desired segments.

Base Address: 0xe0004400

10.1 Segment Bitmask (DATA register, offset 0x00)

Bit	Name	Physical Segment	Description
7	SEG_DP	Decimal point	1 = dot lit
6	SEG_G	Middle bar	1 = lit
5	SEG_F	Top-left bar	1 = lit
4	SEG_E	Bottom-left bar	1 = lit
3	SEG_D	Bottom bar	1 = lit
2	SEG_C	Bottom-right bar	1 = lit
1	SEG_B	Top-right bar	1 = lit
0	SEG_A	Top bar	1 = lit

10.2 Digit Patterns

Digit	Hex Value	Segments Active
0	0x3F	A B C D E F
1	0x06	B C
2	0x5B	A B D E G
3	0x4F	A B C D G
4	0x66	B C F G
5	0x6D	A C D F G
6	0x7D	A C D E F G
7	0x07	A B C
8	0x7F	A B C D E F G
9	0x6F	A B C D F G

10.3 Example — Display the digit "3"

```
    li   t0, 0xe0004400
    li   t1, 0x4F          # digit 3: A+B+C+D+G
    sw   t1, 0(t0)
```

NOTE *The constants SEG7_DIGIT_0 through SEG7_DIGIT_9 are defined in hardware/lib/cs281.inc for use in assembly programs.*

11. Buzzer

The buzzer is a single-bit GPIO output. Writing 1 activates the buzzer; writing 0 deactivates it. To produce a tone, toggle the output rapidly in a loop or timer ISR.

Base Address: 0xe0004000

Bits	Name	Description
31:1	—	Reserved. Always write 0.
0	BUZZER	1 = buzzer ON, 0 = buzzer OFF

11.1 Example — 500 Hz Tone (busy-loop, approximate)

```
    li   t0, 0xe0004000
    li   t2, 1000          # toggle count
.tone_loop:
    li   t1, 1
    sw   t1, 0(t0)         # buzzer ON
    li   a0, 5000          # half-period delay
    call delay
    sw   zero, 0(t0)       # buzzer OFF
    li   a0, 5000
    call delay
    addi t2, t2, -1
    bnez t2, .tone_loop
```

12. Register Quick Reference

All addresses shown are absolute byte addresses. All registers are 32-bit; use lw / sw unless noted.

Address	Name	R/W	Description
0xe0001800	UART_RXTX	R/W	UART data — write byte to TX, read byte from RX
0xe0001804	UART_TXFULL	R	1 = TX FIFO full, do not write
0xe0001808	UART_RXEMPTY	R	1 = RX FIFO empty, no data available
0xe0001810	UART_EV_PENDING	R/W	UART interrupt pending — write 1 to clear
0xe0001814	UART_EV_ENABLE	R/W	UART interrupt enable
0xe0002000	TIMER_LOAD	R/W	Timer load value
0xe0002004	TIMER_RELOAD	R/W	Timer auto-reload (0 = one-shot)
0xe0002008	TIMER_EN	R/W	Timer enable (1=run)
0xe000200C	TIMER_UPDATE	W	Write 1 to latch VALUE
0xe0002010	TIMER_VALUE	R	Current count (latch first)
0xe0002018	TIMER_EV_PENDING	R/W	Timer interrupt pending — write 1 to clear
0xe000201C	TIMER_EV_ENABLE	R/W	Timer interrupt enable
0xe0004000	BUZZER	W	Bit 0: 1=ON, 0=OFF
0xe0004400	SEG7	W	Bits 0-7: segments A-G, DP
0xe0005000	CLINT_MSIP	R/W	Software interrupt pending (bit 0)
0xe0009000	CLINT_MTIMECMP	R/W	Timer compare low word
0xe000BFF8	CLINT_MTIME	R	Current time low word
0xe0015000	GPIO_OUT	R/W	LED outputs — bits 0-3 = LED0-LED3
0xe0015400	GPIO_IN	R	Board inputs — bits 0-3 = BTN0,BTN1,DIP_SW0,DIP_SW1